

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)**

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и сетевые технологии**

Направление подготовки (специальность) **09.03.03 Прикладная информатика**

КУРСОВОЙ ПРОЕКТ

по дисциплине «Мобильная разработка (Android и iOS)»

на тему: Разработка мобильного приложения “Мессенджер”

Обучающаяся Красюк Карина Александровна, К3441

Работа выполнена с оценкой ____

Преподаватель:
Шатинский Григорий Сергеевич

(подпись)

Дата 25.01.2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Создание базовой архитектуры и системы навигации приложения.....	5
1.1 Реализация Single-Activity архитектуры с Navigation Component	5
1.2 Применение Material Design 3 и логирование жизненного цикла	7
2 Реализация архитектурного паттерна MVVM и управление состоянием.....	8
2.1 Проектирование и реализация ViewModel для управления состоянием..	8
2.2 Реактивное обновление UI через LiveData.....	10
3 Интеграция сетевых запросов и локальное кэширование данных	12
3.1 Настройка Retrofit для работы с REST API.....	12
3.2 Реализация локального кэширования через Room	13
3.3 Реализация Repository как единой точки доступа к данным.....	15
4 Улучшение пользовательского интерфейса и реализация фоновой синхронизации.....	19
4.1 Разработка кастомного RecyclerView.Adapter с расширенным функционалом	19
4.2 Реализация фоновой синхронизации через WorkManager	22
4.3 Реализация системы уведомлений и запрос разрешений.....	23
4.4 Обработка системных событий и мониторинг сетевого подключения ..	26
ЗАКЛЮЧЕНИЕ	29

ВВЕДЕНИЕ

В настоящее время мобильные приложения являются основным способом взаимодействия пользователей с цифровыми сервисами. Среди них мессенджеры занимают особое место, обеспечивая быстрое и удобное общение. Разработка такого приложения представляет собой комплексную практическую задачу, которая позволяет освоить ключевые технологии современной Android-разработки.

Создание мессенджера требует не только реализации базовых функций, но и решения важных технических вопросов: организации стабильной работы в условиях нестабильного интернет-соединения, обеспечения мгновенного отклика интерфейса и корректной обработки данных. Современные инструменты разработки для Android, такие как язык Kotlin и библиотеки Jetpack, предоставляют готовые решения для этих задач, что ускоряет процесс создания надежных приложений.

Тема курсовой работы – разработка мобильного приложения “Мессенджер” для платформы Android с использованием современных подходов и библиотек.

Цель работы: создать Android-приложение с архитектурой MVVM, интеграцией REST API, локальным кэшированием, фоновой синхронизацией и уведомлениями.

Задачи:

- 1) Разработать базовую архитектуру и систему навигации (Single-Activity, Navigation Component).
- 2) Реализовать управление состоянием через MVVM с сохранением данных при изменениях конфигурации.
- 3) Интегрировать REST API и локальное кэширование для офлайн-доступа.
- 4) Улучшить пользовательский интерфейс, добавить уведомления и фоновую синхронизацию.

Стек технологий:

- 1) Язык: Kotlin;
- 2) Фреймворк: Android SDK (compileSdk 36, targetSdk 36);
- 3) Архитектура: MVVM (Model-View-ViewModel);
- 4) Библиотеки: Navigation Component (навигация), Material 3 (UI компоненты), Retrofit (сетевые запросы), Room (локальная база данных), Kotlin Coroutines (асинхронность), WorkManager (фоновая синхронизация), LiveData/Flow (реактивные данные), ViewBinding (привязка представлений).

1 Создание базовой архитектуры и системы навигации приложения

1.1 Реализация Single-Activity архитектуры с Navigation Component

Выбрана Single-Activity архитектура с Navigation Component.

Преимущества: единая точка входа, упрощенное управление жизненным циклом, декларативная навигация.

Основная Activity – MainActivity, содержит FragmentContainerView для размещения фрагментов и BottomNavigationView для переключения между экранами.

Листинг 1 – Реализация MainActivity с настройкой навигации

```
class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(TAG, "onCreate")

        binding =
            ActivityMainBinding.inflate(layoutInflater)
            setContentView(binding.root)

        setupNavigation()
        requestPermissions()
    }

    private fun setupNavigation() {
        val navHostFragment = supportFragmentManager
            .findFragmentById(R.id.nav_host_fragment) as
            NavHostFragment
        val navController = navHostFragment.navController

        binding.bottomNavigation.setupWithNavController(navController)
    }
```

}

Навигация настроена через nav_graph.xml, связывающий три фрагмента: NewsFeedFragment, ProfileFragment, SettingsFragment.

На рисунке 1 можно увидеть главный экран приложения с нижней навигацией, демонстрирующий три вкладки: Новости, Профиль, Настройки.

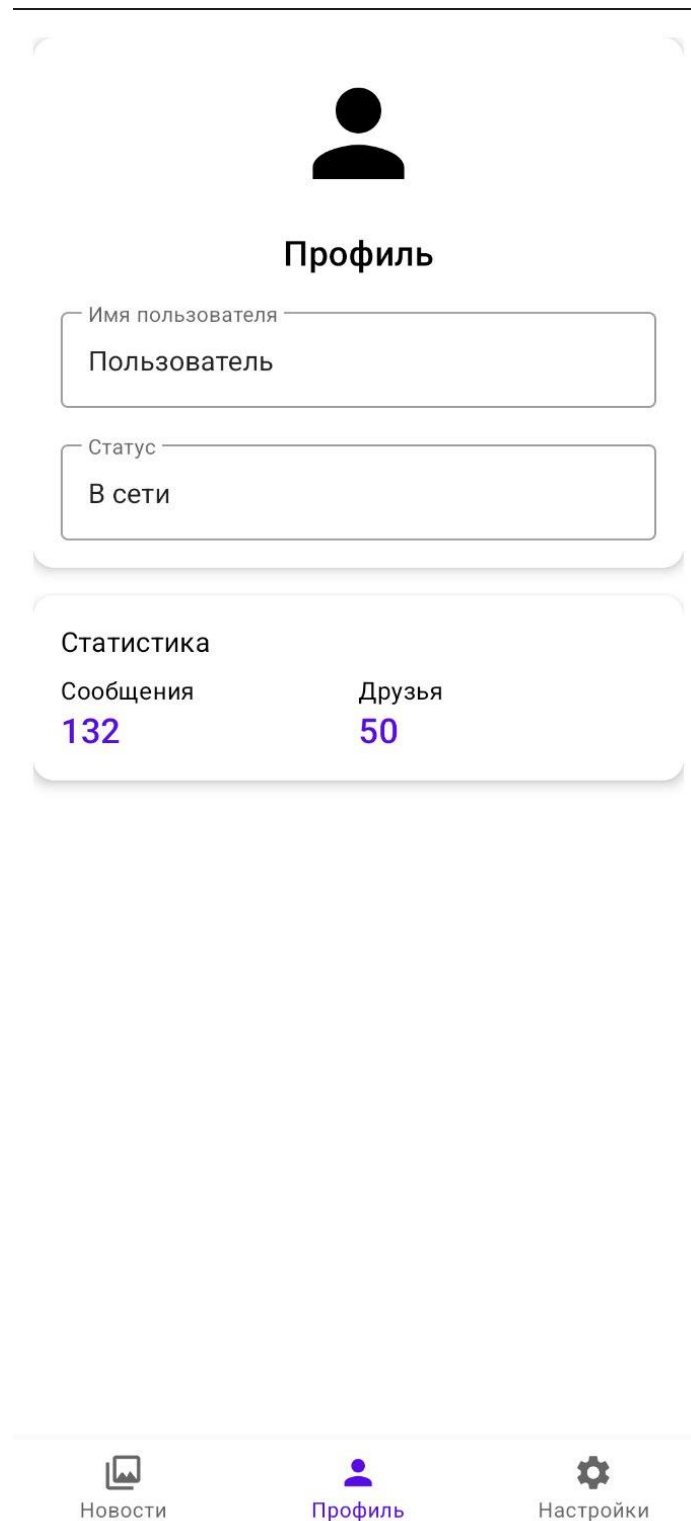


Рисунок 1 – Главный экран приложения с нижней навигацией

1.2 Применение Material Design 3 и логирование жизненного цикла

Применен Material 3 с темой Theme.MaterialComponents.DayNight.NoActionBar для поддержки светлой и темной темы.

Реализовано логирование жизненного цикла Activity и Fragment для отладки. Каждый компонент логирует ключевые события.

Листинг 2 – Логирование жизненного цикла в MainActivity

```
override fun onStart() {  
    super.onStart()  
    Log.d(TAG, "onStart")  
}  
  
override fun onResume() {  
    super.onResume()  
    Log.d(TAG, "onResume")  
}  
  
override fun onPause() {  
    super.onPause()  
    Log.d(TAG, "onPause")  
}  
  
override fun onStop() {  
    super.onStop()  
    Log.d(TAG, "onStop")  
}  
  
override fun onDestroy() {  
    super.onDestroy()  
    Log.d(TAG, "onDestroy")  
}
```

Аналогичное логирование реализовано во всех фрагментах (NewsFeedFragment, ProfileFragment, SettingsFragment).

2 Реализация архитектурного паттерна MVVM и управление состоянием

2.1 Проектирование и реализация ViewModel для управления состоянием

Реализован MVVM с разделением ответственности:

- 1) Model – данные и бизнес-логика;
- 2) View – UI (Activity, Fragment);
- 3) ViewModel – управление состоянием, независимо от жизненного цикла View.

Созданы ViewModel для экранов профиля и настроек.

Листинг 3 – Реализация ProfileViewModel для управления состоянием профиля

```
class ProfileViewModel : ViewModel() {

    companion object {
        private const val TAG = "ProfileViewModel"
    }

    private val _userName = MutableLiveData<String>().apply {
        value = "Пользователь"
    }
    val userName: LiveData<String> = _userName

    private val _userStatus =
MutableLiveData<String>().apply {
        value = "В сети"
    }
    val userStatus: LiveData<String> = _userStatus

    fun updateUserName(name: String) {
        Log.d(TAG, "updateUserName: $name")
        _userName.value = name
    }
}
```



```

    }

    fun updateUserStatus(status: String) {
        Log.d(TAG, "updateUserStatus: $status")
        _userStatus.value = status
    }

    override fun onCleared() {
        super.onCleared()
        Log.d(TAG, "onCleared")
    }

    init {
        Log.d(TAG, "init - ViewModel created")
    }
}

```

Листинг 4 — Реализация SettingsViewModel для управления темой приложения

```

class SettingsViewModel : ViewModel() {

    private val _isDarkTheme =
MutableLiveData<Boolean>().apply {
        value = false
    }

    val isDarkTheme: LiveData<Boolean> = _isDarkTheme

    fun setDarkTheme(isDark: Boolean) {
        Log.d(TAG, "setDarkTheme: $isDark")
        _isDarkTheme.value = isDark

        val themeMode = if (isDark) {
            AppCompatActivity.MODE_NIGHT_YES
        } else {
            AppCompatActivity.MODE_NIGHT_NO
        }
    }
}

```

```

        AppCompatActivity.setDefaultNightMode(themeMode)
    }

    override fun onCleared() {
        super.onCleared()
        Log.d(TAG, "onCleared")
    }

    init {
        Log.d(TAG, "init - ViewModel created")
    }
}

```

ViewModel сохраняют состояние при повороте экрана, так как переживают изменения конфигурации.

2.2 Реактивное обновление UI через LiveData

Использованы LiveData и MutableLiveData для реактивного обновления UI. View подписывается на изменения через observe().

Листинг 5 – Наблюдение за данными во Fragment

```

private val viewModel: ProfileViewModel by viewModels()

override fun onCreateView(view: View, savedInstanceState: Bundle?) {
    super.onCreateView(view, savedInstanceState)

    viewModel.userName.observe(viewLifecycleOwner) { name ->
        binding.editTextUserName.setText(name)
    }

    viewModel.userStatus.observe(viewLifecycleOwner) {
status ->
        binding.editTextUserStatus.setText(status)
    }
}

```

```

binding.buttonSave.setOnClickListener {

viewModel.updateUserName(binding.editTextUserName.text.toString(
))

viewModel.updateUserStatus(binding.editTextUserStatus.text.toStr
ing())

}

}

```

viewLifecycleOwner обеспечивает автоматическую отмену подписки при уничтожении View, предотвращая утечки памяти.

На рисунках 2 и 3 можно увидеть экран профиля с редактируемыми полями имени и статуса пользователя и экран настроек с переключателем темы приложения соответственно.

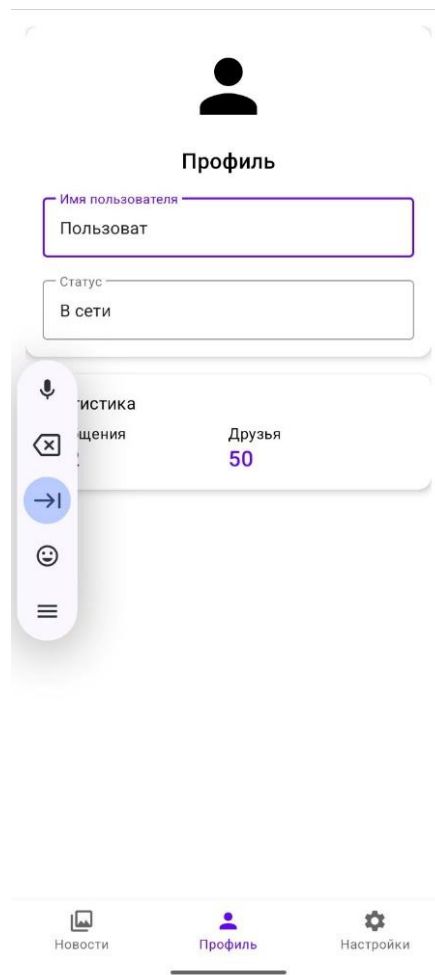


Рисунок 2 – Экран профиля с редактируемыми полями имени и статуса пользователя

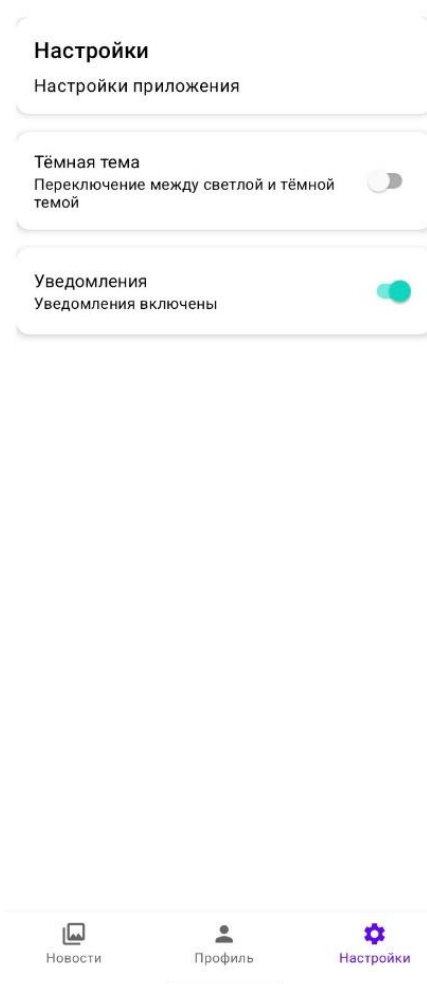


Рисунок 3 – Экран настроек с переключателем темы приложения

3 Интеграция сетевых запросов и локальное кэширование данных

3.1 Настройка Retrofit для работы с REST API

Для сетевых запросов использован Retrofit с корутинами. Создан `NetworkClient` для конфигурации клиента.

Листинг 6 – Конфигурация Retrofit клиента

```
object NetworkClient {  
  
    private const val BASE_URL =  
        "https://jsonplaceholder.typicode.com/"  
  
    private val loggingInterceptor: HttpLoggingInterceptor  
        by lazy {  
            HttpLoggingInterceptor().apply {  
                level = HttpLoggingInterceptor.Level.BASIC  
            }  
        }  
}
```

```

        }
    }

    private val okHttpClient: OkHttpClient by lazy {
        OkHttpClient.Builder()
            .addInterceptor(loggingInterceptor)
            .build()
    }

    private val retrofit: Retrofit by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .client(okHttpClient)

.addConverterFactory(GsonConverterFactory.create())
            .build()
    }

    val messageApi: MessageApiService by lazy {
        retrofit.create(MessageApiService::class.java)
    }
}

```

API-сервис определен через интерфейс с suspend-функциями для работы с корутинами.

Листинг 7 – Определение API-сервиса

```

interface MessageApiService {
    @GET("posts")
    suspend fun getMessages(): List<MessageDto>
}

```

MessageDto – модель данных для десериализации JSON ответов.

3.2 Реализация локального кэширования через Room

Локальное кэширование реализовано через Room. Созданы MessageEntity, MessageDao и AppDatabase.

Листинг 8 – Определение Entity для Room

```

@Entity(tableName = "messages")
data class MessageEntity(
    @PrimaryKey val id: Long,
    val userId: Long,
    val title: String,
    val body: String,
    val isLiked: Boolean = false
)

```

Листинг 9 – Определение DAO интерфейса

```

@Dao
interface MessageDao {

    @Query("SELECT * FROM messages ORDER BY id DESC")
    fun observeMessages(): Flow<List<MessageEntity>>

    @Query("SELECT * FROM messages WHERE id = :messageId")
    suspend fun getMessageById(messageId: Long):
MessageEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(messages: List<MessageEntity>)

    @Query("DELETE FROM messages")
    suspend fun clearAll()

    @Query("UPDATE messages SET isLiked = :isLiked WHERE id
= :messageId")
    suspend fun updateLikeStatus(messageId: Long, isLiked:
Boolean)
}

```

Листинг 10 – Реализация AppDatabase

```

@Database(
    entities = [MessageEntity::class],
    version = 2,
    exportSchema = false
)

```

```

)
abstract class AppDatabase : RoomDatabase() {

    abstract fun messageDao(): MessageDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getInstance(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                INSTANCE ?: Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "messenger_db"
                )
                    .fallbackToDestructiveMigration()
                    .build()
                    .also { INSTANCE = it }
            }
        }
    }
}

```

3.3 Реализация Repository как единой точки доступа к данным

MessageRepository объединяет сетевые запросы и локальное хранилище, обеспечивая единую точку доступа к данным.

Листинг 11 – Реализация MessageRepository

```

class MessageRepository(
    private val db: AppDatabase,
    private val ioDispatcher: CoroutineDispatcher =
Dispatchers.IO
) {

    private val messageDao = db.messageDao()

```

```

        val messages: Flow<List<MessageEntity>> =
messageDao.observeMessages()

suspend fun refreshMessages() =
withContext(ioDispatcher) {
    try {
        val remoteMessages =
NetworkClient.messageApi.getMessages()
        val entities = remoteMessages.map { dto ->
            val existing =
messageDao.getMessageById(dto.id)
            dto.toEntity().copy(isLiked =
existing?.isLiked ?: false)
        }
        messageDao.insertAll(entities)
    } catch (_: Exception) {

    }
}

suspend fun toggleLike(messageId: Long, isLiked:
Boolean) = withContext(ioDispatcher) {
    messageDao.updateLikeStatus(messageId, isLiked)
}
}

```

При отсутствии сети данные загружаются из локальной базы через Flow, обеспечивающий реактивные обновления UI.

Листинг 12 – Наблюдение за данными в NewsFeedViewModel

```

class NewsFeedViewModel(
    private val repository: MessageRepository,
    private val networkMonitor: NetworkMonitor
) : ViewModel() {

    private val _messages =
MutableLiveData<List<MessageEntity>>(emptyList())

```



```

val messages: LiveData<List<MessageEntity>> = _messages

init {
    observeMessages()
}

private fun observeMessages() {
    viewModelScope.launch {
        try {
            repository.messages.collectLatest { list ->
                _messages.postValue(list)
            }
        } catch (e: Exception) {
            _error.value = "Ошибка загрузки данных:
${e.message}"
        }
    }
}

fun refresh() {
    viewModelScope.launch {
        if (networkMonitor.isOnline.value == false) {
            _error.value = "Нет подключения к интернету"
            return@launch
        }
        _isLoading.value = true
        _error.value = null
        try {
            repository.refreshMessages()
        } catch (e: Exception) {
            _error.value = e.message ?: "Неизвестная
ошибка"
        } finally {
            _isLoading.value = false
        }
    }
}

```

```

    }
}
}

```

На рисунке 4 можно наблюдать экран новостей со списком сообщений, загруженных из API и отображаемых через RecyclerView.

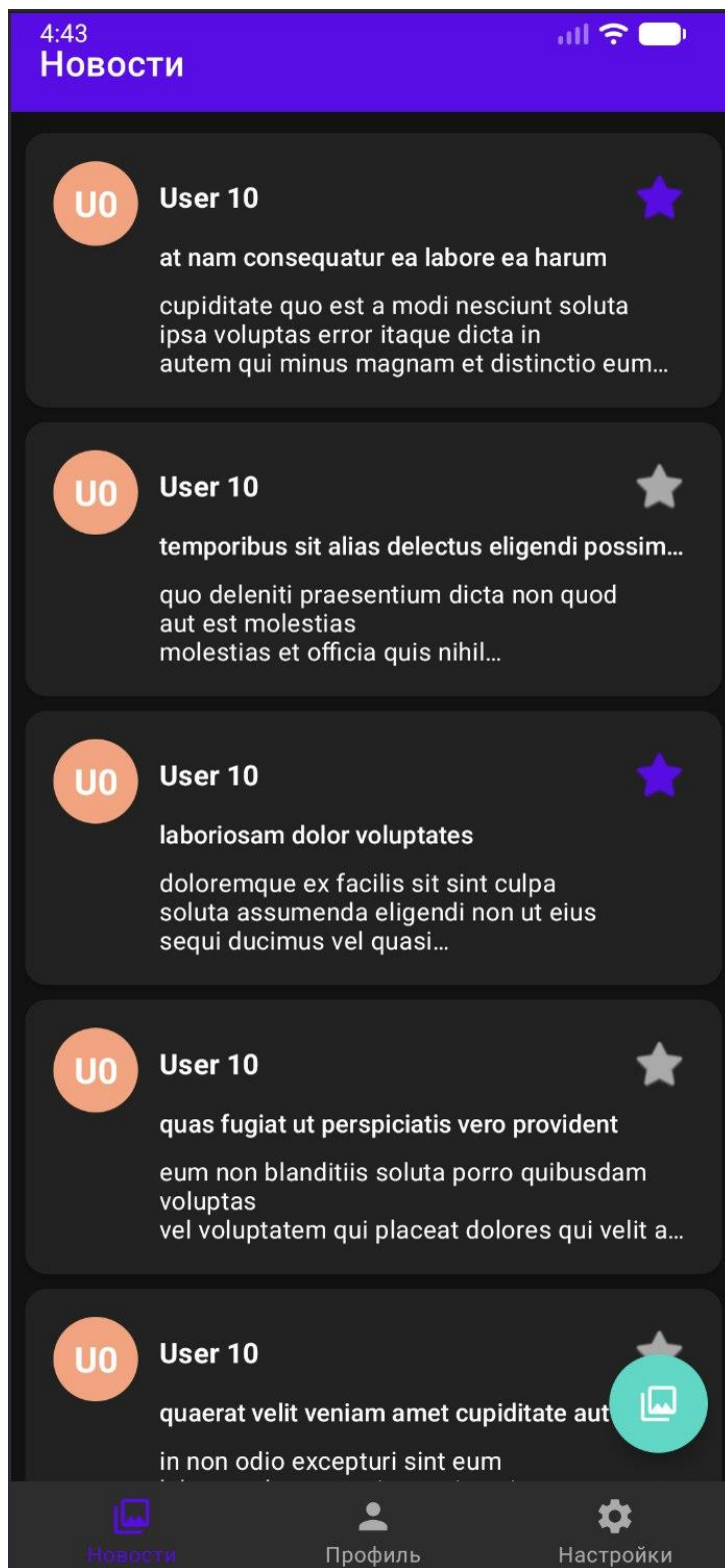


Рисунок 4 – Экран новостей со списком сообщений

4 Улучшение пользовательского интерфейса и реализация фоновой синхронизации

4.1 Разработка кастомного RecyclerView.Adapter с расширенным функционалом

Реализован кастомный RecyclerView.Adapter с аватарками, именами пользователей и функционалом лайков.

Листинг 13 – Реализация MessageAdapter

```
class MessageAdapter(  
    private val onLikeClick: (Long, Boolean) -> Unit  
    ) : ListAdapter<MessageEntity,  
MessageAdapter.MessageViewHolder>(DiffCallback) {  
  
    override fun onCreateViewHolder(parent: ViewGroup,  
viewType: Int): MessageViewHolder {  
        val inflater = LayoutInflater.from(parent.context)  
        val binding = ItemMessageBinding.inflate(inflater,  
parent, false)  
        return MessageViewHolder(binding, onLikeClick)  
    }  
  
    override fun onBindViewHolder(holder: MessageViewHolder,  
position: Int) {  
        holder.bind(getItem(position))  
    }  
  
    class MessageViewHolder(  
        private val binding: ItemMessageBinding,  
        private val onLikeClick: (Long, Boolean) -> Unit  
    ) : RecyclerView.ViewHolder(binding.root) {  
  
        fun bind(item: MessageEntity) {  
            binding.textName.text = "User ${item.userId}"  
            binding.textTitle.text = item.title  
            binding.textBody.text = item.body  
        }  
    }  
}
```

```

        val avatarColor = getAvatarColor(item.userId)

binding.imageAvatar.setBackgroundColor(avatarColor)
        binding.imageAvatar.setImageDrawable(null)
        binding.textAvatarLetter.text = "U${item.userId
% 10}"

        updateLikeButton(item.isLiked)

        binding.buttonLike.setOnClickListener {
            val newLikeState = !item.isLiked
            onLikeClick(item.id, newLikeState)
        }
    }

    private fun updateLikeButton(isLiked: Boolean) {
        val context = binding.root.context
        if (isLiked) {

binding.buttonLike.setImageResource(android.R.drawable.btn_star_
big_on)

            binding.buttonLike.imageTintList =
ContextCompat.getColorStateList(
                context,
                R.color.purple_500
            )
        } else {

binding.buttonLike.setImageResource(android.R.drawable.btn_star_
big_off)

            binding.buttonLike.imageTintList =
ContextCompat.getColorStateList(
                context,
                android.R.color.darker_gray

```

```

        )
    }
}

private fun getAvatarColor(userId: Long): Int {
    val colors = intArrayOf(
        Color.parseColor("#FF6B6B"),
        Color.parseColor("#4ECDC4"),
        Color.parseColor("#45B7D1"),
        Color.parseColor("#FFA07A"),
        Color.parseColor("#98D8C8"),
        Color.parseColor("#F7DC6F"),
        Color.parseColor("#BB8FCE")
    )
    return colors[(userId % colors.size).toInt()]
}

private object DiffCallback :
DiffUtil.ItemCallback<MessageEntity>() {
    override fun areItemsTheSame(oldItem: MessageEntity,
newItem: MessageEntity): Boolean =
        oldItem.id == newItem.id

    override fun areContentsTheSame(oldItem:
MessageEntity, newItem: MessageEntity): Boolean =
        oldItem == newItem
}
}

```

Использованы компоненты Material Design:

- 1) CardView – карточки сообщений;
- 2) FloatingActionButton – кнопка обновления;
- 3) AppBarLayout – панель инструментов.

4.2 Реализация фоновой синхронизации через WorkManager

Фоновая синхронизация реализована через WorkManager с периодическим выполнением каждые 15 минут.

Листинг 14 – Реализация SyncWorker

```
class SyncWorker(  
    context: Context,  
    params: WorkerParameters  
) : CoroutineWorker(context, params) {  
  
    override suspend fun doWork(): Result {  
        return try {  
            val db =  
AppDatabase.getInstance(applicationContext)  
            val repository = MessageRepository(db)  
  
            repository.refreshMessages()  
  
NotificationHelper.showSyncNotification(applicationContext)  
  
            Result.success()  
        } catch (e: Exception) {  
            Result.retry()  
        }  
    }  
}
```

Листинг 15 – Настройка периодической синхронизации

```
object WorkManagerHelper {  
    private const val SYNC_WORK_NAME = "sync_messages_work"  
  
    fun scheduleSync(context: Context) {  
        val constraints = Constraints.Builder()  
            .setRequiredNetworkType(NetworkType.CONNECTED)  
            .build()  
    }  
}
```

```

        val syncWorkRequest =
PeriodicWorkRequestBuilder<SyncWorker>(
            15, TimeUnit.MINUTES
        )
            .setConstraints(constraints)
            .build()

WorkManager.getInstance(context).enqueueUniquePeriodicWork(
    SYNC_WORK_NAME,
    ExistingPeriodicWorkPolicy.KEEP,
    syncWorkRequest
)
    }
}

```

4.3 Реализация системы уведомлений и запрос разрешений

Уведомления реализованы через NotificationHelper с созданием канала уведомлений.

Листинг 16 – Реализация NotificationHelper

```

object NotificationHelper {
    private const val CHANNEL_ID = "sync_channel"
    private const val CHANNEL_NAME = "Синхронизация"
    private const val NOTIFICATION_ID = 1

    fun createNotificationChannel(context: Context) {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O)
        {
            val channel = NotificationChannel(
                CHANNEL_ID,
                CHANNEL_NAME,
                NotificationManager.IMPORTANCE_DEFAULT
            ).apply {
                description = "Уведомления о синхронизации
данных"
            }
        }
    }
}

```

```

        val notificationManager =

context.getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager

notificationManager.createNotificationChannel(channel)
        }
    }

    fun showSyncNotification(context: Context) {
        val intent = Intent(context,
MainActivity::class.java).apply {
            flags = Intent.FLAG_ACTIVITY_NEW_TASK or
Intent.FLAG_ACTIVITY_CLEAR_TASK
        }
        val pendingIntent = PendingIntent.getActivity(
            context,
            0,
            intent,
            PendingIntent.FLAG_IMMUTABLE or
PendingIntent.FLAG_UPDATE_CURRENT
        )

        val notification =
NotificationCompat.Builder(context, CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_news)

.setContentTitle(context.getString(R.string.sync_notification_title))

.setContentText(context.getString(R.string.sync_notification_text))

.setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .setContentIntent(pendingIntent)

```



```

        .setAutoCancel(true)
        .build()

    val notificationManager =

context.getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager

        notificationManager.notify(NOTIFICATION_ID,
notification)
    }
}

```

Листинг 17 – Запрос разрешений в MainActivity

```

private fun requestPermissions() {
    val permissionsToRequest = mutableListOf<String>()

    if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.TIRAMISU) {
        if (ContextCompat.checkSelfPermission(
            this,
            Manifest.permission.POST_NOTIFICATIONS
        ) != PackageManager.PERMISSION_GRANTED
        ) {

permissionsToRequest.add(Manifest.permission.POST_NOTIFICATIONS)

        }

        if (ContextCompat.checkSelfPermission(
            this,
            Manifest.permission.READ_CONTACTS
        ) != PackageManager.PERMISSION_GRANTED
        ) {

permissionsToRequest.add(Manifest.permission.READ_CONTACTS)

        }
    }
}

```

```

        if (permissionsToRequest.isNotEmpty()) {

requestPermissionLauncher.launch (permissionsToRequest.toTypedArr
ay())

        } else {
            initializeBackgroundServices()
        }
    }

private fun initializeBackgroundServices() {
    NotificationHelper.createNotificationChannel(this)
    WorkManagerHelper.scheduleSync(this)
}

```

4.4 Обработка системных событий и мониторинг сетевого подключения

Реализован NetworkMonitor для отслеживания состояния сети и реакции на изменения подключения.

Листинг 18 – Реализация NetworkMonitor

```

class NetworkMonitor(private val context: Context) {
    private val connectivityManager =

context.getSystemService (Context.CONNECTIVITY_SERVICE) as
ConnectivityManager

    private val _isOnline = MutableLiveData<Boolean>()
    val isOnline: LiveData<Boolean> = _isOnline

    private val networkCallback = object :
ConnectivityManager.NetworkCallback() {
        override fun onAvailable(network: Network) {
            _isOnline.postValue(true)
        }
    }
}

```

```

        override fun onLost(network: Network) {
            _isOnline.postValue(false)
        }

        override fun onCapabilitiesChanged(
            network: Network,
            networkCapabilities: NetworkCapabilities
        ) {
            val hasInternet =
networkCapabilities.hasCapability(
                NetworkCapabilities.NET_CAPABILITY_INTERNET
            ) && networkCapabilities.hasCapability(
                NetworkCapabilities.NET_CAPABILITY_VALIDATED
            )
            _isOnline.postValue(hasInternet)
        }
    }

    init {
        checkInitialState()
        registerNetworkCallback()
    }

    private fun checkInitialState() {
        val network = connectivityManager.activeNetwork
        val capabilities = network?.let {
connectivityManager.getNetworkCapabilities(it) }
        val isConnected =
capabilities?.hasCapability(NetworkCapabilities.NET_CAPABILITY_I
NTERNET) == true
        &&
capabilities.hasCapability(NetworkCapabilities.NET_CAPABILITY_VA
LIDATED)

        _isOnline.value = isConnected
    }

```

```

        private fun registerNetworkCallback() {
            val networkRequest = NetworkRequest.Builder()

.addCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
            .build()

connectivityManager.registerNetworkCallback(networkRequest,
networkCallback)
        }

        fun unregister() {

connectivityManager.unregisterNetworkCallback(networkCallback)
        }
    }

```

Листинг 19 – Использование NetworkMonitor в NewsFeedFragment

```

viewModel.isOnline.observe(viewLifecycleOwner) { isOnline ->
    if (!isOnline) {
    } else {
        if (adapter.itemCount == 0) {
            viewModel.refresh()
        }
    }
}

```

На рисунке 5 можно увидеть уведомление о успешной синхронизации данных, отображаемое в системной панели уведомлений.

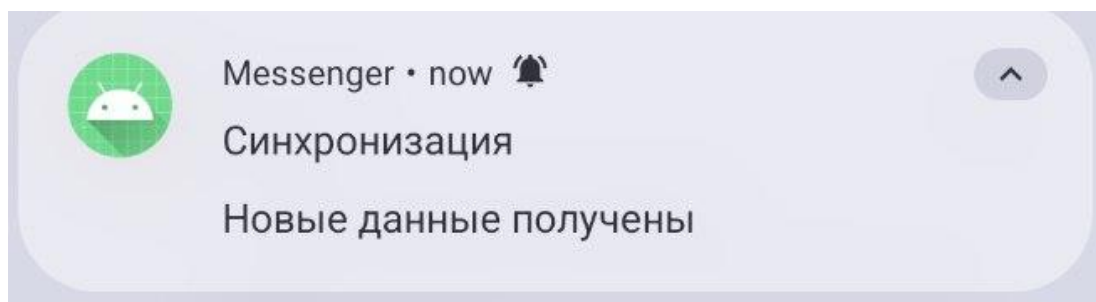


Рисунок 5 – Уведомление о успешной синхронизации данных

ЗАКЛЮЧЕНИЕ

В рамках выполнения курсовой работы было разработано и протестировано мобильное приложение «Мессенджер» для платформы Android. Проект успешно реализован с применением актуального стека технологий и современных архитектурных подходов, что позволило создать стабильное и функциональное приложение.

В ходе работы были последовательно выполнены следующие ключевые этапы:

- 1) Создана базовая структура Single-Activity с Navigation Component и Material 3.
- 2) Реализован MVVM с управлением состоянием через ViewModel и LiveData.
- 3) Интегрирован REST API через Retrofit и локальное кэширование через Room.
- 4) Улучшен UI с кастомным адаптером, Material Design компонентами и функционалом лайков.
- 5) Реализована фоновая синхронизация через WorkManager и система уведомлений.
- 6) Добавлена обработка системных событий (онлайн/офлайн) через NetworkMonitor.

Цель достигнута: приложение демонстрирует применение современных практик Android-разработки, включая MVVM, корутины, Room, WorkManager и Material Design.

Итоговая функциональность приложения включает:

- 1) Навигация между экранами через Navigation Component;
- 2) Сохранение состояния при изменениях конфигурации через ViewModel;
- 3) Загрузка данных из REST API с локальным кэшированием;
- 4) Офлайн-доступ к данным через Room;
- 5) Фоновая синхронизация через WorkManager;

- 6) Уведомления о синхронизации;
- 7) Реакция на изменения сетевого подключения.

Таким образом, работа носит законченный характер. Созданный прототип мессенджера является полностью рабочим, соответствует современным стандартам качества. Выполненный проект подтверждает готовность к решению комплексных задач в области мобильной разработки.